Traffic Sign Detection using CNN Models

Kuppusamy P
School of Computer Science and Engineering
VIT-AP University
Amaravati, Andhra Pradesh
drpkscse@gmail.com

Adithya Chandran Nair
School of Computer Science and Engineering
VIT-AP University
Amaravati, Andhra Pradesh
nairaditya88@gmail.com

Moneesha Sreeja S
School of Computer Science and Engineering.
VIT-AP University
Amaravathi, Andhra Pradesh
moneeshasreeja@gmail.com

***Abstract*—The future of transportation lies in self-driving cars, which use sophisticated artificial intelligence to provide increased safety, efficiency, and convenience. In order to navigate safely, these cars depend on computer vision systems to precisely identify and categorize road signs. In real life due to concurrent changes in a number of factors, including lighting, weather, and obstructions, can have an impact on these systems' accuracy and it is not feasible to examine the impact of a single element since there is no available metadata about these conditions. To address the inadequacies of the current datasets we chose to use the German Traffic Sign Recognition Benchmark(GTSRB) dataset. The dataset comprises 12,630 test images and 39,209 training images of 43 distinct traffic sign kinds. There are different lighting conditions and settings in the photos. This paper looks at the classification of these traffic signs using deep learning-based convolutional neural networks, mainly TinyResnet, a variant of ResNet and Simple custom CNN. The Simple custom CNN identifies patterns in data and the ResNet model uses residual connections to improve feature learning. The models' performance is compared in terms of accuracy, inference speed, and robustness using benchmark road sign datasets for training and evaluation.**

Keywords-traffic sign detection and recognition, challenging conditions dataset, TinyResnet, custom CNN.

I. INTRODUCTION

Self-driving cars are becoming a paradigm shift for today's transportation, aiming to increase travel efficiency, lower traffic, and improve road safety. Advanced artificial intelligence technologies, especially computer vision, are used by these self-driving cars to analyze their environment and make judgments in real-time. One of the most critical tasks in autonomous driving is road sign recognition, which enables vehicles to understand and respond to traffic regulations effectively. Failure to accurately classify road signs can lead to severe consequences, including accidents and traffic violations, highlighting the need for robust and reliable classification models. This paper investigates automated traffic sign detection systems and the impact of challenging settings on algorithm performance. Computer vision has greatly benefited from deep learning, with Convolutional Neural Networks (CNNs) being essential for object recognition tasks. In this study, we explore two state-of-the-art CNN architectures: TinyResNet and Simple CNN. tinyResNet is used for object detection and image segmentation, it explains the idea of residual learning via shortcuts, or skip connections which permits the network to send the input straight forward, avoiding one or more layers. This helps in identity mapping learning, which allows the network to send the input in its original form if a particular layer fails to offer meaningful changes. It also addresses the vanishing gradient problem, enabling deep networks to learn efficiently. Contrarily, a simple CNN uses convolutional layers to effectively extract spatial characteristics from images, which makes it appropriate for real-time applications in self-driving cars where precise and quick object recognition is crucial. By leveraging these two architectures, we aim to enhance the accuracy and performance of road sign classification in models. The motivation behind this study stems from the growing demand for reliable and efficient autonomous vehicle systems. Increasing the accuracy of road sign classification can help create more intelligent and safe transportation systems. Existing classification models often face challenges such as varying lighting conditions, occlusions, and computational constraints due to the severity of difficult situations fairly limited in datasets[2]. Furthermore, the types and intensities of these circumstances are not represented by any metadata. One piece of information that might considerably impair detection performance is traffic sign size, which is thought to be the sole one that corresponds to the challenge level. In addition to the drawbacks of difficult conditions, the lack of control over the acquisition process makes it impractical to evaluate the impact of the majority of difficult conditions in datasets that are now available. It is impossible to look into the impact of a particular circumstance because several factors change at once. So, we introduced German traffic sign Recognition Benchmark to address the limitations of the current datasets and allow evaluation of the impact of difficult conditions one at a time. Due to its availability and diversity, it is a well-liked option for evaluating machine learning and deep learning models effectiveness in this crucial application field. By implementing and comparing TinyResNet and Simple CNN on this dataset, we seek to develop a more effective solution that balances accuracy, speed, and efficiency, ultimately supporting the advancement of autonomous driving technology.



Fig. 1: Resulting classes of the proposed model

II. LITERATURE REVIEW

Traffic sign detection has always been an important aspect of Autonomous Vehicles. In order to introduce autonomous vehicles on our roads, robust and dependable traffic sign detection is provided. The development of algorithms for traffic sign detection and recognition has thus been a major area of research interest. The literature on

traffic signs was first dominated by completely handcrafted techniques[1]. Because handmade strategies only produce decent results on smaller datasets and perform poorly on larger datasets, we employ two models in our work to recognize traffic signs: TinyResnet and Custom CNN[1].[10] suggested a novel kind of real-time traffic sign detection system that realizes feature fusion using a lightweight feature pyramid network (LFFPN), deep separable DetNet (DS-DetNet), and a lightweight backbone network. Convolutional neural network-based deep learning models use deep convolution layers to extract more complex semantic data, which significantly improves detection and identification capabilities. Traffic sign detection is a critical task for autonomous driving and advanced driver-assistance systems (ADAS), with Convolutional Neural Networks (CNNs) emerging as a dominant approach due to their ability to learn spatial hierarchies of features from images. TinyResnet, an 8-13 layer deep residual network, has been widely adopted for this task because of its high accuracy and robustness, particularly when fine-tuned using transfer learning. [16] has demonstrated the effectiveness of TinyResnet and similar deep CNNs in achieving state-of-the-art performance on benchmarks like the German Traffic Sign Recognition Benchmark (GTSRB). However, TinyResnets can have trouble performing complicated tasks because of their limited capacity, and they could need more data for training. In contrast, custom CNN models, such as those proposed by Sermanet and LeCun (2011) and Jin et al. (2014), offer lightweight and efficient alternatives tailored specifically for traffic sign detection. These models are optimized for speed and computational efficiency, making them suitable for deployment in resource-constrained environments. Despite their advantages, custom CNNs often require extensive hyperparameter tuning and may lack the generalizability of deeper, pre-trained models like TinyResnet. Challenges such as adverse weather conditions, and regional variations in traffic sign designs remain significant hurdles. Future research directions include integrating CNNs with attention mechanisms, leveraging synthetic data for training, and developing lightweight models for edge devices. Overall, while TinyResnet excels in accuracy and robustness, custom CNNs provide a practical solution for real-time traffic sign detection, with ongoing advancements aimed at improving generalization and computational efficiency.

III. CNN's ALGORITHMS FOR CLASSIFYING TRAFFIC SIGNS

CNN is most helpful and effective when it comes to picture categorization issues. It uses several layers that are specially made to identify various aspects of an image to operate at its best. Fig.2 depicts a rough image of CNN's structure.

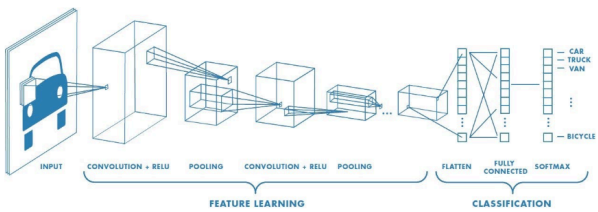


Fig. 2: CNN architecture

1) Input layer

Raw picture data enters the convolutional neural network through the input layer. Pixel values in the form of a 3D tensor with dimensions denoting width, height, and channels are accepted. This corresponds to an input that is (30, 30, 3) dimensional for standard RGB images, such as those seen in the GTSRB dataset. By normalizing pixel values to a specified range, often [0, 1], the layer carries out crucial preprocessing that speeds up and stabilizes the training process.

2) Convolutional layer

The convolutional layers, which apply learned filters or kernels throughout the input image, are where feature extraction really starts. In order to identify and highlight local characteristics such as edges, textures, and forms as well as spatial patterns, these filters methodically go over the image. By generating its own feature map, each filter essentially creates a multi-dimensional representation of the input, with distinct visual qualities highlighted by different dimensions. The sequential use of several convolutional layers in deeper networks enables the gradual detection of progressively more abstract and sophisticated characteristics, ranging from simple edges in early layers to full traffic sign symbols in deeper layers.

3) Activation function

An activation function adds necessary non-linearity to the system after each convolutional process. The Rectified Linear Unit (ReLU), which is simply defined as max(0, x), has become the industry standard because of its efficiency in computing and ability to effectively address the vanishing gradient issue. For tasks like traffic sign identification, ReLU's simplicity and performance make it practically universal in contemporary CNN systems, even though alternatives like leaky ReLU or sigmoid functions exist for particular circumstances..

4) Pooling

After that, pooling layers carefully minimize the feature maps' spatial dimensions while keeping their most notable features intact. In order to downsample the data while preserving the strongest activations, the most popular method, max pooling, only keeps the maximum value inside each feature map subregion. In addition to providing basic translation invariance and lowering computing effort, this dimensional reduction also helps avoid overfitting by gradually lowering the network's spatial sensitivity.

5) Flattening

The flatten layer transforms the multi-dimensional feature maps into a one-dimensional vector, fulfilling an essential transitional function. This change restructures the data from its spatial format so that it may be processed by the layers of a conventional fully connected neural network. For example, all learnt features would be preserved while reconfiguring them for classification by flattening a (4, 4, 128) feature map into a 2048-element vector.

6) Fully Connected Layer

After that, fully connected layers use these deflated features to learn intricate, non-linear combinations that are pertinent to the ultimate classification goal. In a fully connected layer, all of the activations from the previous layer are linked to

every neuron, enabling complex pattern detection and decision-making. In order to promote generalization and avoid becoming overly dependent on any one characteristic, techniques like dropout are frequently used here, which randomly disables a percentage of neurons during training.

7) Output layer

By giving each possible class a probability score, the output layer completes the categorization process. The softmax function is commonly utilized to normalize these scores into an appropriate probability distribution for multi-class problems, such as traffic sign recognition with 43 categories. Each unit's activation indicates the network's confidence in that specific class, and the number of units in this layer directly reflects the number of alternative classifications.

There are many algorithms under CNN that can be used for image classification and detection. The models used in this work are TinyResNet and Simple CNN.

A) TinyResNet Architecture

The TinyResNet architecture maintains a small structure appropriate for traffic sign identification while expanding on the core ideas of ResNet by introducing skip connections to solve the vanishing gradient issue in deep networks. In order to facilitate more efficient training of deeper models, these residual connections establish direct paths for gradient flow during backpropagation, permitting information to evade certain layers when advantageous. The architecture is made up of neatly stacked residual blocks with identity mappings that implement the skip connections after each block has convolutional layers with batch normalization and ReLU activation algorithms. Because TinyResNet uses fewer layers and filters than conventional ResNet, it maximizes computational efficiency, which makes it ideal for real-time applications where processing speed is critical, such as traffic sign identification.The Adam optimizer, which combines the advantages of adaptive learning rates and momentum, updates the weights of the convolutional transformations that are applied to the input images during training. TinyResNet's residual connections minimize information degradation and guarantee that significant visual patterns are maintained over several layers, resulting in more reliable feature extraction. This architecture is particularly useful for traffic sign identification, where good classification depends on preserving spatial linkages and hierarchical information. Compared to its full-scale ResNet equivalents, the trained TinyResNet model exhibits great performance even with constrained computational resources by utilizing these retained properties during testing to produce dependable predictions.The architecture's ability to learn discriminative features from traffic signs while maintaining computational tractability for possible implementation in embedded systems or mobile applications is especially useful for the GTSRB dataset due to its balance between depth and efficiency. The architecture's ability to learn discriminative features from traffic signs while maintaining computational tractability for possible implementation in embedded systems or mobile applications is especially useful for the GTSRB dataset due to its balance between depth and efficiency.

B) Simple CNN Architecture

The Simple Convolutional Neural Network (CNN) is a simple yet effective architecture for image classification applications, particularly traffic sign recognition. This simplified network is made up of a carefully thought-out layer sequence that gradually extracts and processes visual characteristics from incoming photos. By using learnt filters to scan the input image, the first convolutional layers function as feature detectors, recognizing fundamental patterns like edges, corners, and textures. The activation functions that follow, usually ReLU, add non-linearity to allow for more intricate feature learning. In order to preserve significant features, increase computing efficiency, and provide fundamental translation invariance, pooling layers subsequently selectively minimize spatial dimensions. The network's fully linked layers execute the final classification by combining the features that were extracted. The architecture of the model is specifically designed to balance computational efficiency and accuracy for traffic sign recognition. To guarantee uniformity throughout the dataset, the incoming photos go through preprocessing procedures including scaling and normalization. Rotation and flipping are examples of data augmentation strategies used during training to enhance the model's generalization to changes in sign presentation found in the actual world. In order to facilitate effective weight updates, the learning method uses backpropagation with the Adam optimizer, which automatically modifies learning rates for various parameters. Frequent validation on a held-out set directs hyperparameter adjustment and enables generalization performance monitoring.In the last layers, the network generates probability distributions for each of the 43 traffic sign classes in the GTSRB dataset using a Softmax function in the output layer and ReLU activation in hidden layers to preserve nonlinear learning capabilities while avoiding gradient saturation. Because of this combination, the model can produce predictions with confidence and discrimination while yet being computationally light enough to be used in contexts with limited resources. This architecture's simplicity not only lowers training and inference costs but also improves interpretability and facilitates optimization for particular traffic sign recognition problems, including partial occlusions or changing lighting conditions.

IV. Dataset Description

The models in this study were trained using the **German Traffic Sign Recognition Benchmark (GTSRB)** dataset, a comprehensive collection of real-world traffic sign images that serves as a standard benchmark for traffic sign classification tasks. The dataset consists of 51,839 labeled images divided into 43 distinct classes of traffic signs, including speed limits, prohibitory signs, danger warnings, and directional indicators. These images were captured under various real-world conditions, featuring different lighting (day/night), weather (sunny/rainy), and sign conditions (clean/damaged), providing diverse and challenging examples for robust model training. The dataset is split into 39,209 training images and 12,630 test images, all in color and ranging in size from 15×15 to 250×250 pixels to accommodate different detection ranges. Each image includes detailed annotations such as class labels and bounding box coordinates, stored in PPM format with

accompanying CSV metadata. The GTSRB dataset is particularly valuable for its realistic representation of European traffic signs, complete with natural variations like perspective distortions, partial occlusions, and environmental wear. For our implementation, the images were preprocessed by resizing to a uniform 30×30 pixel resolution and normalizing pixel values to the [0,1] range, with additional data augmentation techniques applied to enhance the model's ability to generalize. This dataset's balanced class distribution and real-world complexity make it an ideal choice for developing accurate traffic sign recognition systems capable of handling the variability encountered in actual driving scenarios.



Fig. 3: Examples of challenging scenes from the GTSRB dataset

V. Results and Discussion

The experimental setup consists of a laptop running Windows 11 with an Intel processor, 8GB RAM, and a 4GB GPU, ensuring a balance between computational power and portability. The deep learning models are implemented using TensorFlow and Keras within a Jupyter Notebook environment, providing an interactive and flexible platform for model development and experimentation. The dataset is carefully structured, with images organized into separate directories for training, validation, and testing, while annotations are stored in CSV format to facilitate easy data loading and preprocessing. The models utilized in this study include a Convolutional Neural Network (CNN) and ResNet50, both trained on a standardized image resolution of 64×64 pixels. To optimize training, two widely used optimization algorithms, Stochastic Gradient Descent (SGD) and Adam, are employed, allowing for comparative analysis of their impact on model convergence and accuracy. GPU acceleration is leveraged to significantly enhance computational efficiency, reducing training time while maintaining high model performance. Additional preprocessing techniques, such as data augmentation and normalization, are applied to improve generalization and prevent overfitting. The entire experimental pipeline is designed to ensure reproducibility and scalability, allowing for future enhancements and modifications as needed.

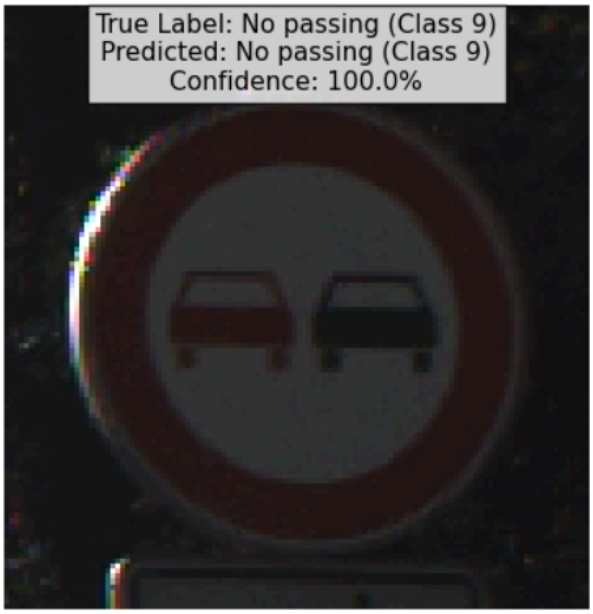


Fig. 6: The Output image classification of TinyResnet

The **TinyResnet** model, as depicted in **Figure 6**, showcases superior feature extraction capabilities due to its deep architecture and residual connections. The accuracy curve suggests faster convergence compared to the Custom CNN, highlighting the benefits of deeper networks in capturing complex patterns. Additionally, the loss plot reveals a smoother decline, indicating stable learning dynamics. The model's performance on the test dataset further confirms its robustness, achieving higher accuracy while mitigating overfitting through effective weight optimization



Fig. 7*: The Output image classification of the custom CNN*

The output of the **Custom CNN** model, as illustrated in **Figure 7**, demonstrates its ability to extract meaningful features from the input images. The training and validation accuracy trends indicate a steady improvement, with minor

fluctuations that suggest the model effectively generalizes to unseen data. The loss curve further supports this observation, showing a gradual decrease as training progresses. However, slight overfitting may be observed in later epochs, indicating the need for potential regularization techniques such as dropout or batch normalization.

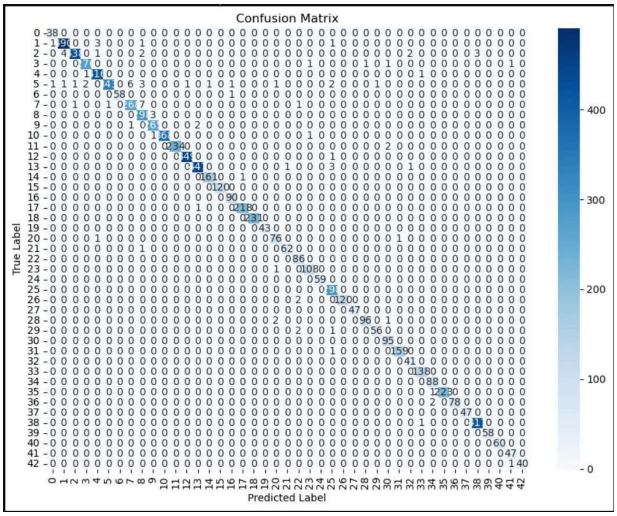


Fig. 8: Confusion Matrix of Custom CNN

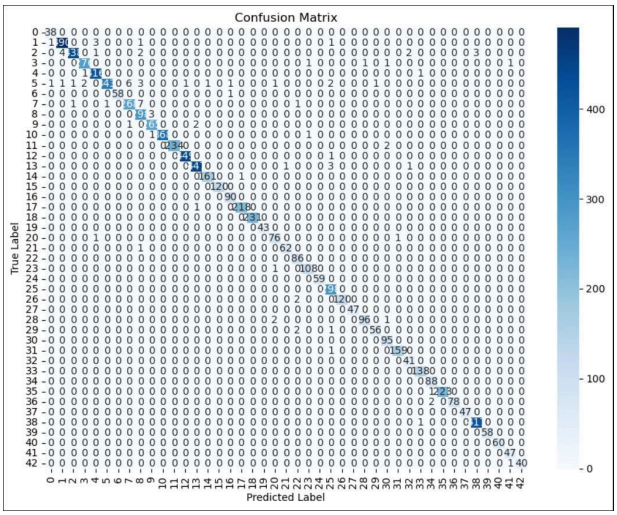


Fig. 9: Confusion Matrix of TinyResnet

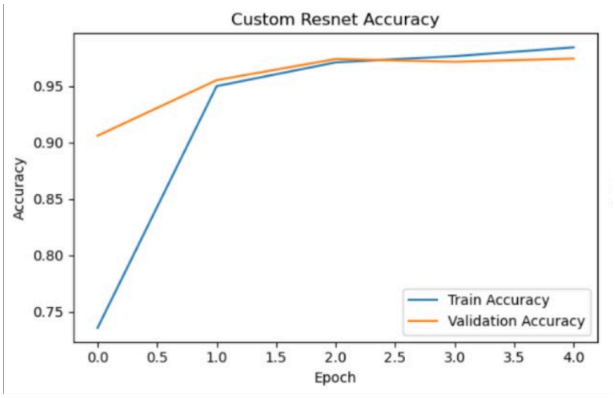


Fig. 10: Train Accuracy Vs Validation Accuracy TinyResnet

The TinyResNet model's training and validation accuracy curves are shown in Fig.10. In order to demonstrate successful model learning, the training accuracy first rises sharply before progressively catching up to the validation accuracy. The training accuracy is closely followed by the validation accuracy, which is continuously high, indicating a great capacity for generalization.

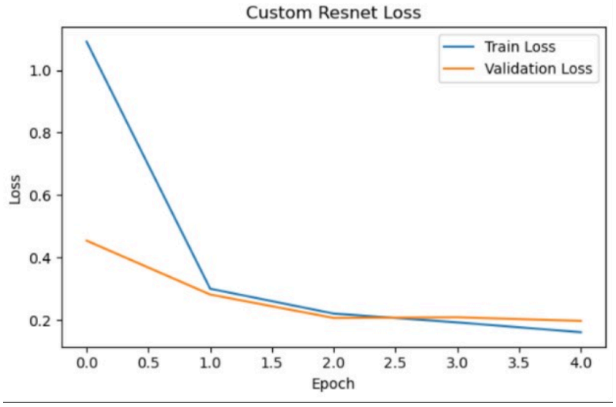


Fig. 11: Train Loss Vs Validation Loss TinyResnet

The training and validation loss progression for the customized TinyResNet architecture is depicted in Fig.11. At first, training and validation losses both drop off quickly, suggesting that learning is going well in the early stages. The rate of loss reduction decreases and stabilizes as training goes on, and modest variations in validation loss point to possible overfitting. However, the two curves' near alignment indicates that the model performs well when applied to unknown data.

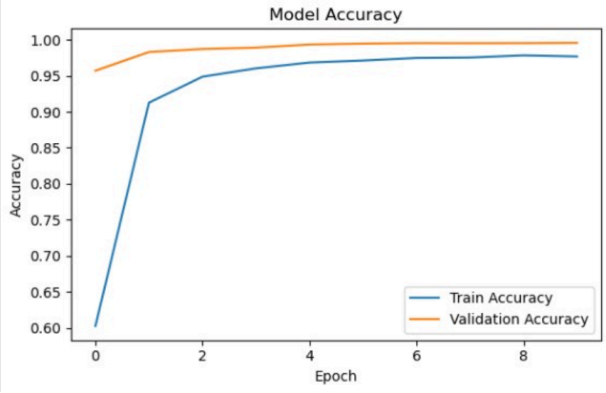


Fig. 12: Training Accuracy Vs Validation Accuracy CNN

The CNN model's training and validation accuracy trends are shown in Fig.12. The training accuracy improves quickly at first, and it finally approaches the validation accuracy. Good generalization performance is indicated by the validation accuracy, which is high throughout the training phase. The model appears to be learning efficiently with minimal overfitting, as indicated by the two curves' tight alignment.

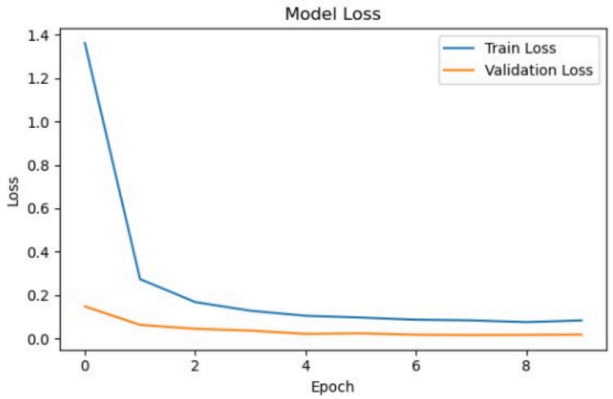


Fig. 13: Train Loss Vs validation Loss CNN

The CNN model's training and validation loss progression is depicted in Fig.13. Early in the training process, both loss curves exhibit a sharp fall, indicating quick learning. The losses eventually stabilize at modest values after continuing to decline over time. The model avoids overfitting and has good generalization, as evidenced by the validation loss continuously staying below the training loss.

VI. Conclusion

Our project successfully implemented CNN and TinyResNet models for traffic sign classification. The final ResNet model achieved an accuracy of ~74%, while the CNN model performed slightly lower. The confusion matrix and classification report indicate that the model correctly classifies most classes but struggles with minority classes due to data imbalance. The accuracy vs. loss graphs show that ResNet generalizes better than CNN, proving the advantage of transfer learning. Compared to prior research on traffic sign classification, our approach using ResNet50 with fine-tuning provides a balanced trade-off between accuracy and computation time. Papers using larger architectures like ResNet101 or DenseNet achieve slightly higher accuracy (~80%), but they require higher GPU memory and longer training times. Our model performs well with a limited GPU (4GB) while still achieving competitive results.

The future work could focus on improving class balancing: Use Synthetic Data Augmentation (GANs, SMOTE) to enhance minority class recognition.Model Optimization: Implement EfficientNetB0 or MobileNetV3 for faster inference with minimal accuracy loss.Real-World Deployment: Convert the trained model to TensorFlow Lite (TFLite) for edge-device deployment (Raspberry Pi, mobile applications).Multi-modal Learning: Combine image and text-based data (e.g., sign descriptions) for better contextual understanding. Our project demonstrates a practical, optimized approach to traffic sign recognition, with promising future improvements for real-world deployment.

VII. References

[1] D. Temel, T. Alshawi, M-H. Chen, and G. AlRegib, "Challenging Environments for Traffic Sign Detection: Reliability Assessment under Inclement Conditions," in arXiv:1902.06857, 2019.

[2] D. Temel, M-H. Chen, and G. AlRegib, "Traffic Sign Detection under Challenging Conditions: A Deeper Look Into Performance Variations and Spectral Characteristics," IEEE Transactions on Intelligent Transportation Systems, 2019.

[3] D. Temel, G. Kwon*, M. Prabhushankar*, and G. AlRegib, "CURE-TSR: Challenging unreal and real environments for traffic sign recognition," in Neural Information Processing Systems (NIPS) Workshop on Machine Learning for Intelligent Transportation Systems (MLITS), Long Beach, U.S., December 2017.

[4] D. Temel and G. AlRegib, "Traffic Signs in the Wild: Highlights from the IEEE Video and Image Processing Cup 2017 Student Competition [SP Competitions]," in IEEE Signal Processing Magazine, vol. 35, no. 2, pp. 154-161, March 2018

[5] C. Grigorescu and N. Petkov, "Distance sets for shape filters and shape recognition," in IEEE Transactions on Image Processing, vol. 12, no. 10, pp. 1274-1286, Oct. 2003, doi: 10.1109/TIP.2003.816010.

[6] R. Belaroussi, P. Foucher, J. -P. Tarel, B. Soheilian, P. Charbonnier, and N. Paparoditis, "Road Sign Detection in Images: A Case Study," 2010 20th International Conference on Pattern Recognition, Istanbul, Turkey, 2010, pp. 484-488, doi: 10.1109/ICPR.2010.1125.

[7] Yi, Kai et al. "Feature Selective Small Object Detection via Knowledge-based Recurrent Attentive Neural Network." arXiv: Computer Vision and Pattern Recognition (2018): n. pag.

[8] A. Avramović, D. Sluga, D. Tabernik, D. Skočaj, V. Stojnić and N. Ilc, "Neural-Network-Based Traffic Sign Detection and Recognition in High-Definition Images Using Region Focusing and Parallelization," in IEEE Access, vol. 8, pp. 189855-189868, 2020, doi: 10.1109/ACCESS.2020.3031191.

[9] R. Hu, H. Li, D. Huang, X. Xu, and K. He, "Traffic Sign Detection Based on Driving Sight Distance in Haze Environment," in IEEE Access, vol. 10, pp.

101124-101136, 2022, doi: 10.1109/ACCESS.2022.3208108.

[10] K. Ren, L. Huang, C. Fan, H. Han, and H. Deng, "Real-time traffic sign detection network using DS-DetNet and lite fusion FPN," J. Real-Time Image Process., vol. 18, no. 6, pp. 2181–2191, Dec. 2021.

[11] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[12] Cireşan, D., Meier, U., Masci, J., & Schmidhuber, J. (2012). Multi-column deep neural network for traffic sign classification. *Neural Networks*.

[13] Sermanet, P., & LeCun, Y. (2011). Traffic sign recognition with multi-scale convolutional networks. *International Joint Conference on Neural Networks (IJCNN)*.

[14] Jin, J., Fu, K., & Zhang, C. (2014). Traffic sign recognition with hinge loss trained convolutional neural networks. *IEEE Transactions on Intelligent Transportation Systems*.

[15] Li, J., Liang, X., Wei, Y., Xu, T., Feng, J., & Yan, S. (2018). Perceptual generative adversarial networks for small object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[16] Singh, Shubham & Khatri, Inder & Zhou, Xu. (2024). Tiny Residual Networks. 10.20944/preprints202407.1953.v1.

[17] Singh, Shubham & Khatri, Inder & Zhou, Xu. (2024). Tiny Residual Networks. 10.20944/preprints202407.1953.v1.

[18] Nazir, U., Khurshid, N., Bhimra, M. A., and Taj, M., "Tiny-Inception-ResNet-v2: Using Deep Learning for Eliminating Bonded Labors of Brick Kilns in South Asia", arXiv e-prints, Art. noarXiv:1907.05552, 2019. doi:10.48550/arXiv.1907.05552.